Bab 3 Perancangan dan Implementasi Basis Data Lanjut

Tujuan Pembelajaran

Setelah mempelajari bab ini, Anda mampu:

1. **(C2 – Memahami)** menjelaskan konfigurasi integrasi basis data, mekanisme eksekusi query, dan pemanfaatan Eloquent ORM pada Laravel.
2. **(C3 – Menerapkan)** mengimplementasikan migrasi, seeder, model, serta query menggunakan Query Builder dan Eloquent untuk skema yang lebih kompleks.
3. **(C4 – Menganalisis)** menganalisis relasi antarentitas dan strategi pemetaan objek-relasional terhadap integritas data serta performa query.
4. **(C6 – Mencipta)** merancang pemetaan objek-relasional yang meliputi model, relasi, migrasi, aturan unik, dan data uji untuk domain bisnis multi-pihak.

Orientasi Bab

Pada Bab 2, Anda telah mempelajari cara request bergerak melalui route, middleware, controller, service, dan view. Namun, sebagian besar fitur aplikasi tidak akan berjalan tanpa data yang tersimpan secara konsisten.

Aplikasi *Kantin Multi-Tenant* harus menyimpan menu, kategori, tenant, pesanan, pembayaran, dan data pendukung lain. Data tersebut tidak cukup hanya "tersimpan". Struktur basis data harus mencegah relasi yang rusak, membatasi data antar-tenant, mendukung perubahan skema, dan tetap dapat diakses dengan efisien.

Bab ini membahas proses menerjemahkan rancangan basis data menjadi implementasi Laravel. Kita akan bergerak dari ERD ke migrasi, dari tabel ke model Eloquent, dari relasi ke query, lalu ke data uji dan verifikasi performa dasar.

> **Peta bab.** Bab ini menggunakan ERD aplikasi *Kantin Multi-Tenant* sebagai skema acuan. Contoh data, perhitungan, dan skenario bersifat **ilustratif** untuk pembelajaran.

[PERLU VERIFIKASI: kesesuaian contoh dengan ERD baseline Fase 0, SRS, dan Modul Praktikum Kantin Multi-Tenant Laravel 13]

> **Keterkaitan mata kuliah.** Konsep *Entity Relationship Diagram* (ERD), normalisasi, kunci primer, dan kunci asing dibahas pada mata kuliah *Basis Data* (RPL01016). Pembahasan transaksi, indeks, dan optimasi lebih lanjut terdapat pada *Basis Data Lanjut* (RPL01029). Bab ini berfokus pada **implementasi** rancangan tersebut dalam Laravel.

3.1 Mengapa Implementasi Basis Data Perlu Dirancang

Basis data bukan hanya tempat menyimpan baris data. Basis data juga membantu menegakkan aturan yang penting bagi aplikasi. Misalnya, pada aplikasi *Kantin Multi-Tenant*:

- satu menu harus terkait dengan tenant yang tepat;
- nama menu mungkin harus unik dalam satu tenant;
- pesanan tidak boleh merujuk menu yang tidak ada;
- penghapusan tenant tidak boleh dilakukan tanpa mempertimbangkan data transaksi;

- halaman katalog harus tetap responsif ketika jumlah menu bertambah.

Keputusan tersebut perlu diterjemahkan ke dalam beberapa unsur implementasi.

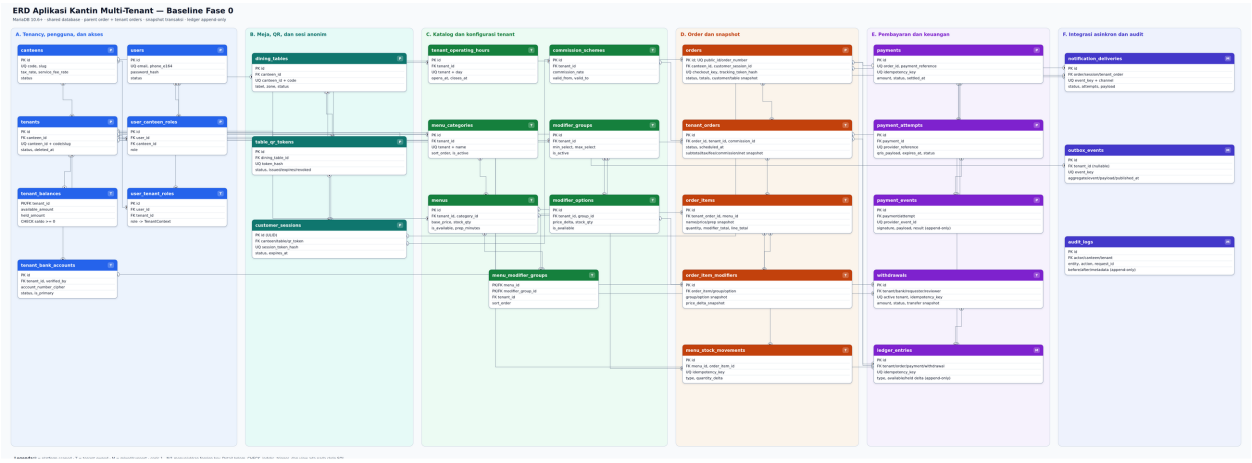
Kebutuhan	Implementasi utama
Menyimpan struktur tabel	Migration
Menjaga hubungan antarentitas	Foreign key dan aturan penghapusan
Menjaga aturan unik	Unique index atau validasi tambahan
Mengakses data sebagai objek	Model Eloquent dan relasi
Menampilkan data secara efisien	Query Builder, Eloquent, eager loading, indeks
Menyediakan data pengembangan	Factory dan seeder
Memeriksa hasil implementasi	Migrasi, pengujian data, dan EXPLAIN

Tabel 3.1. Kebutuhan aplikasi dan unsur implementasi basis data yang menjawabnya.

Dengan demikian, implementasi basis data perlu dipandang sebagai proses yang terhubung: ERD → migration → model → relasi → query → data uji → verifikasi.

3.2 Dari ERD ke Skema Laravel

ERD menggambarkan entitas, atribut, dan hubungan data pada tingkat konseptual atau logis. Laravel membutuhkan bentuk implementasi yang lebih konkret: nama tabel, kolom, tipe data, kunci, indeks, dan aturan penghapusan. Gambar 3.1 menjadi acuan rancangan basis data aplikasi *Kantin Multi-Tenant*.



Gambar 3.1. ERD acuan aplikasi Kantin Multi-Tenant sebagai skema sasaran implementasi.

Sumber: artefak desain basis data Fase 0 (SRS), ilustratif.

3.2.1 Menerjemahkan Entitas dan Relasi

Tabel 3.2 menunjukkan cara umum menerjemahkan unsur ERD menjadi implementasi basis data Laravel.

Unsur ERD	Implementasi Laravel/MariaDB	Contoh
Entitas	Tabel dan model Eloquent	menus dan Menu
Identitas entitas	Primary key	$table->id()
Atribut teks	Kolom string() atau text()	$table->string('nama', 120)

Nilai uang Rupiah	Bilangan bulat tanpa pecahan	$table->unsignedBigInteger('harga')
Relasi banyak-ke-satu	Foreign key dan belongsTo()	menus.tenant_id
Relasi satu-ke-banyak	hasMany() pada model induk	Tenant::menus()
Relasi banyak-ke-banyak	Tabel penghubung atau pivot	order_items
Aturan unik	Unique index	unique(['tenant_id', 'nama'])
Akses yang sering disaring	Index	indeks pada kolom tenant atau status

Tabel 3.2. Penerjemahan unsur ERD menjadi implementasi Laravel.

Sebelum membuat migration, Anda perlu menjawab pertanyaan desain berikut.

1. Apa identitas unik setiap data?
2. Data mana yang wajib ada?
3. Tabel mana yang menjadi induk dan tabel mana yang menjadi anak?
4. Apa yang terjadi jika data induk dihapus?
5. Kolom mana yang sering digunakan untuk mencari, menyaring, atau menggabungkan data?
6. Apakah suatu aturan bisnis harus ditegakkan pula oleh basis data?

Pertanyaan tersebut membantu Anda tidak hanya membuat tabel yang “bisa berjalan”, tetapi tabel yang melindungi integritas data.

3.3 Konfigurasi Koneksi dan Akses Basis Data

Laravel membutuhkan informasi koneksi untuk berkomunikasi dengan basis data. Setelah koneksi tersedia, aplikasi dapat menggunakan Query Builder, Eloquent ORM, atau SQL mentah secara terukur.

3.3.1 Konfigurasi Environment dan Koneksi

Konfigurasi basis data lazim dipisahkan ke berkas environment `.env`. Laravel kemudian membaca nilai tersebut melalui berkas konfigurasi, misalnya `config/database.php`.

Parameter	Makna
DB_CONNECTION	Nama koneksi atau driver basis data
DB_HOST	Alamat server basis data
DB_PORT	Nomor porta server basis data
DB_DATABASE	Nama basis data
DB_USERNAME	Nama pengguna basis data
DB_PASSWORD	Kata sandi pengguna basis data
charset	Kumpulan karakter yang digunakan
collation	Aturan perbandingan dan pengurutan teks

Tabel 3.3. Parameter konfigurasi koneksi basis data.

Berikut adalah contoh konseptual konfigurasi koneksi MariaDB.

Kode 3.1. Kutipan konfigurasi koneksi basis data — config/database.php

```php
<?php

return [
```

```
    'default' => env('DB_CONNECTION', 'mariadb'),

    'connections' => [
        'mariadb' => [
            'driver'    => 'mariadb',
            'host'      => env('DB_HOST', '127.0.0.1'),
            'port'      => env('DB_PORT', '3306'),
            'database'  => env('DB_DATABASE', 'kantin'),
            'username'  => env('DB_USERNAME', 'kantin_user'),
            'password'  => env('DB_PASSWORD', ''),
            'charset'   => 'utf8mb4',
            'collation' => 'utf8mb4_unicode_ci',
            'strict'    => true,
        ],
    ],
];
```

Pada contoh tersebut:

- `utf8mb4` digunakan agar aplikasi mampu menyimpan karakter Unicode secara lebih lengkap;
- `strict` digunakan untuk membantu basis data menolak sebagian data yang tidak sesuai aturan;
- nilai seperti host, nama basis data, dan kredensial dibaca dari environment.

Peringatan. Jangan menyimpan kredensial nyata pada repository. Simpan `.env` di lingkungan lokal atau server, gunakan `.env.example` tanpa rahasia, dan berikan akun basis data hanya hak akses yang diperlukan.

[PERLU VERIFIKASI: driver `mariadb`, struktur konfigurasi, serta pengaturan default yang tersedia pada versi Laravel proyek]

Untuk memeriksa status migrasi, Anda dapat menggunakan perintah berikut.

```
Terminal pada root proyek
php artisan migrate:status
```

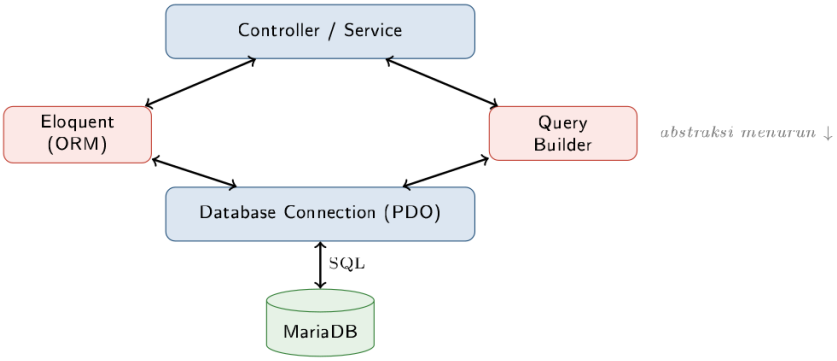
Perintah tersebut membantu melihat apakah aplikasi dapat mengakses tabel migrasi dan migration mana yang telah dijalankan.

3.3.2 Lapisan Akses Basis Data

Laravel menyediakan beberapa cara untuk mengakses basis data. Ketiganya berjalan di atas koneksi PDO yang sama, tetapi berbeda tingkat abstraksinya (Gambar 3.2).

Pendekatan	Karakter utama	Cocok digunakan ketika
Eloquent ORM	Tingkat abstraksi tinggi; menggunakan model dan relasi	CRUD, relasi domain, dan kode aplikasi sehari-hari
Query Builder	Menyusun query secara fluent	Agregasi, join, laporan, dan query terstruktur
Raw SQL	Kendali SQL penuh	Query khusus yang sulit diungkapkan melalui builder atau ORM

Tabel 3.4. Tiga lapisan akses basis data pada Laravel.



Gambar 3.2. Alur akses basis data: controller atau service, lapisan abstraksi, PDO, lalu MariaDB.

Sumber: diadaptasi penulis, ilustratif.

Eloquent dan Query Builder mendukung *parameter binding*. Binding memisahkan nilai parameter dari sintaks SQL sehingga membantu mencegah injeksi SQL ketika API digunakan dengan benar. Namun, keamanan tidak otomatis dijamin hanya karena Anda menggunakan Eloquent atau Query Builder.

> **Peringatan.** Hindari membangun SQL dengan menggabungkan string dari request pengguna. Penggunaan `DB::raw()`, `whereRaw()`, atau SQL mentah tetap harus diaudit dan menggunakan binding parameter bila menerima nilai dinamis.

3.3.3 Query Builder untuk Rekap Data

Query Builder cocok untuk query agregasi atau laporan yang tidak selalu perlu dibentuk sebagai objek model lengkap.

Kode 3.2. Rekap stok menu tersedia per tenant

```php
use Illuminate\Support\Facades\DB;

$rekap = DB::table('menus')
    ->select('tenant_id', DB::raw('SUM(stok) AS total_stok'))
    ->where('tersedia', true)
    ->groupBy('tenant_id')
    ->orderBy('tenant_id')
    ->get();
```

Kode tersebut melakukan beberapa hal.

1. `DB::table('menus')` memilih tabel `menus`.
2. `where('tersedia', true)` menyaring menu yang tersedia.
3. `SUM(stok)` menghitung total stok.
4. `groupBy('tenant_id')` membuat hasil agregasi per tenant.
5. `get()` menjalankan query dan mengambil hasilnya.

Pada contoh ini, `DB::raw()` hanya berisi ekspresi SQL statis. Jangan memasukkan nilai dari request pengguna ke dalam string `DB::raw()`.

Metode	Fungsi
where() dan orWhere()	Menyaring baris berdasarkan kondisi
join() dan leftJoin()	Menggabungkan tabel

groupBy() dan having()	Mengelompokkan serta menyaring hasil agregasi
orderBy() dan limit()	Mengurutkan dan membatasi hasil
count(), sum(), dan avg()	Menghitung nilai agregat

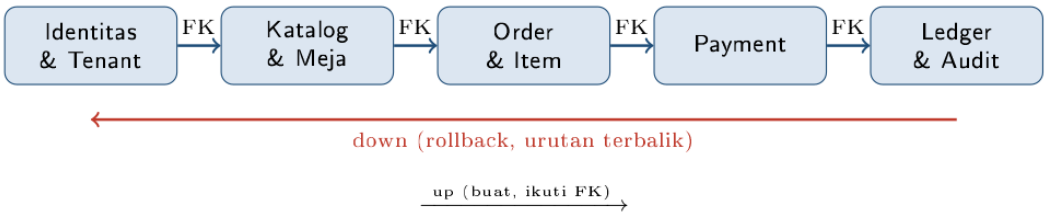
Tabel 3.5. Metode Query Builder yang sering digunakan.

3.4 Migrasi dan Integritas Skema

Migration adalah kendali versi untuk struktur basis data. Dengan migration, setiap anggota tim dapat membangun skema yang sama dari kode yang sama.

3.4.1 Urutan Migration dan Dependensi Tabel

Tabel anak yang memiliki foreign key harus dibuat setelah tabel induknya tersedia. Tabel `menus` tidak dapat dibuat dengan foreign key ke `tenants` dan `menu_categories` jika kedua tabel induk tersebut belum tersedia. Saat rollback, prosesnya berlaku sebaliknya: tabel anak perlu dihapus atau diubah terlebih dahulu sebelum tabel induknya.



Gambar 3.3. Urutan migration berdasarkan dependensi foreign key.

Sumber: diadaptasi penulis, ilustratif.

Nama berkas migration biasanya memiliki awalan *timestamp*. Awalan tersebut membantu Laravel menentukan urutan eksekusi. Namun, timestamp tidak menggantikan pemahaman tentang dependensi tabel. Anda tetap perlu memeriksa foreign key, urutan migration, dan skenario rollback.

3.4.2 Tipe Data dan Aturan Bisnis

Pemilihan tipe data merupakan keputusan desain. Tipe data harus mencerminkan karakter data serta aturan bisnis yang ingin dijaga.

Tipe Laravel	Padanan umum MariaDB	Penggunaan
id()	BIGINT UNSIGNED	Primary key
foreignId()	BIGINT UNSIGNED	Foreign key
string(n)	VARCHAR(n)	Teks pendek dengan batas panjang
text()	TEXT	Teks lebih panjang
unsignedBigInteger('harga')	BIGINT UNSIGNED	Uang dalam Rupiah bulat
unsignedInteger('stok')	INT UNSIGNED	Stok yang tidak bernilai negatif
boolean()	TINYINT(1)	Nilai benar atau salah
timestamps()	Kolom waktu	Waktu pembuatan dan pembaruan data
unique([...])	Unique index komposit	Aturan nilai unik pada gabungan

		kolom

Tabel 3.6. Tipe kolom migration dan pertimbangan penggunaannya.

Harga menu dapat disimpan sebagai bilangan bulat Rupiah, misalnya 15000, bukan sebagai *float*. Pendekatan tersebut membantu menghindari masalah representasi pecahan biner pada nilai uang.

3.4.3 Membuat Tabel Menu

Kode 3.3. Migration tabel menus

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('menus', function (Blueprint $table) {
            $table->id();

            $table->foreignId('tenant_id')
                ->constrained()
                ->restrictOnDelete();

            $table->foreignId('category_id')
                ->constrained('menu_categories')
                ->restrictOnDelete();

            $table->string('nama', 120);
            $table->unsignedBigInteger('harga');
            $table->unsignedInteger('stok')->default(0);
            $table->boolean('tersedia')->default(true);
            $table->timestamps();

            $table->unique(['tenant_id', 'nama']);
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('menus');
    }
};
```

Migration tersebut menegakkan beberapa aturan.

- Setiap menu memiliki tenant.
- Setiap menu memiliki kategori.
- Harga dan stok tidak menggunakan nilai negatif pada tingkat tipe data.
- Nama menu unik dalam satu tenant.
- Menu tidak dapat tetap ada jika tenant atau kategori yang dirujuk dihapus secara sembarangan.

Contoh menggunakan `restrictOnDelete()` agar penghapusan tenant atau kategori yang masih dipakai menu tidak langsung menghapus data turunan. Pilihan ini bukan aturan universal, melainkan keputusan desain yang bergantung pada kebijakan data.

Strategi penghapusan	Kapan dipertimbangkan
cascadeOnDelete()	Data anak memang harus hilang bersama induknya dan tidak diperlukan untuk riwayat
restrictOnDelete()	Penghapusan induk harus dicegah jika masih memiliki data terkait
nullOnDelete()	Relasi boleh dikosongkan dan kolom foreign key dapat bernilai null
Nonaktifkan atau arsipkan data	Riwayat perlu dipertahankan untuk audit atau pelaporan

Tabel 3.7. Strategi penghapusan data induk dan kondisi penggunaannya.

Peringatan. `cascadeOnDelete()` dapat menghapus banyak data secara berantai. Pada sistem transaksi, keputusan tersebut harus mengikuti kebijakan retensi data, audit, dan kebutuhan pelaporan. Jangan menerapkannya hanya untuk mempermudah penghapusan data.

Aturan unik `unique(['tenant_id', 'nama'])` berarti dua tenant dapat memiliki menu bernama "Kopi Susu", tetapi satu tenant tidak dapat menyimpan dua menu dengan nama yang sama. Pastikan aturan ini sesuai dengan kebutuhan bisnis dan perilaku *collation* basis data terhadap perbedaan huruf besar-kecil.

3.4.4 Migration sebagai Riwayat Perubahan

Setelah migration dipakai oleh tim atau dijalankan pada lingkungan produksi, hindari mengubah isi migration lama. Buat migration baru untuk menambah, mengubah, atau menghapus struktur. Sebagai contoh, jika Anda ingin menambah deskripsi menu, buat migration baru.

```
Terminal pada root proyek
php artisan make:migration add_deskripsi_to_menus_table
```

Pendekatan ini menjaga riwayat perubahan skema dan membantu seluruh lingkungan aplikasi bergerak secara konsisten.

3.5 Model Eloquent dan Relasi Domain

Eloquent ORM memetakan tabel basis data menjadi objek PHP. Model menyediakan tempat untuk mengatur atribut yang boleh diisi, konversi tipe, serta relasi data.

3.5.1 Model Menu

Kode 3.4. Model Eloquent Menu

```php
<?php

namespace App\Modules\Catalog\Models;

use App\Modules\Tenant\Models\Tenant;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\BelongsToMany;

class Menu extends Model
{
    protected $fillable = [
        'category_id',
        'nama',
        'harga',
        'stok',
        'tersedia',
    ];
```

```php
    protected $casts = [
        'harga'    => 'integer',
        'stok'     => 'integer',
        'tersedia' => 'boolean',
    ];

    public function tenant(): BelongsTo
    {
        return $this->belongsTo(Tenant::class);
    }

    public function category(): BelongsTo
    {
        return $this->belongsTo(MenuCategory::class, 'category_id');
    }

    public function modifiers(): BelongsToMany
    {
        return $this->belongsToMany(Modifier::class, 'menu_modifier_groups');
    }
}
```

Model tersebut memiliki beberapa fungsi.

- `$fillable` menentukan atribut yang boleh diisi melalui *mass assignment*.
- `$casts` memastikan nilai tertentu dibaca dan ditulis dengan tipe yang diharapkan.
- `tenant()` dan `category()` menyatakan bahwa satu menu dimiliki oleh satu tenant dan satu kategori.
- `modifiers()` menyatakan relasi *many-to-many* melalui tabel pivot.

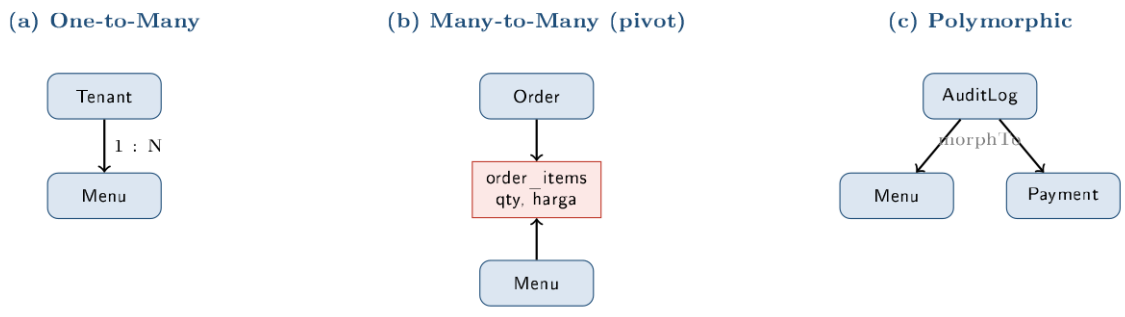
Atribut `tenant_id` sengaja tidak dimasukkan dalam `$fillable` pada contoh ini. Pada aplikasi multi-tenant, tenant sebaiknya ditentukan dari konteks server yang tepercaya, misalnya middleware atau service, bukan dari nilai yang langsung dikirim pengguna melalui formulir.

Peringatan. `$fillable` bukan satu-satunya pertahanan keamanan. Anda tetap harus memvalidasi input dan memeriksa otorisasi. Namun, membatasi atribut yang dapat diisi membantu mengurangi risiko perubahan kolom sensitif secara tidak sengaja.

[PERLU VERIFIKASI: namespace Tenant, MenuCategory, dan Modifier; nama tabel menu_modifier_groups; foreign key; serta kardinalitas relasi berdasarkan ERD proyek]

3.5.2 Relasi One-to-Many

Relasi *one-to-many* digunakan ketika satu entitas memiliki banyak entitas lain. Contohnya, satu tenant memiliki banyak menu. Gambar 3.4 merangkum bentuk relasi yang dipakai pada bab ini.



Gambar 3.4. Bentuk relasi antarmodel yang dipakai pada domain Kantin Multi-Tenant.

Sumber: diadaptasi penulis, ilustratif.

Kode 3.5. Deklarasi relasi one-to-many pada kedua sisi

```php
// Pada model Tenant
public function menus(): HasMany
{
    return $this->hasMany(Menu::class);
}

// Pada model Menu
public function tenant(): BelongsTo
{
    return $this->belongsTo(Tenant::class);
}
```

Dengan relasi tersebut, Anda dapat mengambil seluruh menu milik tenant melalui `$menus = $tenant->menus;`.

3.5.3 Relasi Many-to-Many dan Pivot Beratribut

Relasi *many-to-many* digunakan ketika banyak data pada tabel pertama dapat terhubung dengan banyak data pada tabel kedua. Contoh konseptualnya adalah pesanan dan menu: satu pesanan dapat berisi banyak menu, dan satu menu dapat muncul pada banyak pesanan.

Namun, tabel `order_items` biasanya menyimpan data tambahan seperti kuantitas, harga saat dipesan, catatan pelanggan, atau subtotal. Karena itu, tabel pivot tidak hanya menjadi penghubung, tetapi juga menyimpan informasi transaksi.

Kode 3.6. Relasi many-to-many dengan pivot beratribut

```php
public function menus(): BelongsToMany
{
    return $this->belongsToMany(Menu::class, 'order_items')
        ->withPivot(['jumlah', 'harga_satuan'])
        ->withTimestamps();
}
```

Harga pada `order_items` lazim diperlakukan sebagai harga *snapshot*. Artinya, harga yang tersimpan pada item pesanan merekam harga ketika pesanan dibuat. Jika harga katalog berubah pada kemudian hari, riwayat pesanan sebelumnya tetap konsisten.

Jika tabel pivot memiliki banyak aturan bisnis, status, perhitungan, atau perilaku khusus, pertimbangkan untuk memodelkannya sebagai entitas tersendiri, misalnya `OrderItem`, bukan hanya sebagai pivot pasif.

3.5.4 Relasi Polymorphic

Relasi polimorfik memungkinkan satu tabel berelasi dengan beberapa jenis model. Salah satu contoh penggunaannya adalah audit log: satu `AuditLog` dapat mencatat perubahan pada menu, pesanan, atau pembayaran.

Kode 3.7. Relasi polimorfik pada model AuditLog

```php
public function auditable(): MorphTo
{
    return $this->morphTo();
}
```

Relasi ini fleksibel dan dapat mengurangi jumlah tabel audit terpisah. Namun, relasi polimorfik juga memiliki keterbatasan: foreign key biasa tidak selalu dapat menegakkan integritas referensial untuk

target yang dapat berasal dari banyak tabel. Gunakan relasi polimorfik ketika fleksibilitas target memang dibutuhkan dan strategi audit telah dirancang — bukan untuk menghindari perancangan relasi yang spesifik.

3.6 Query Efisien dan Pencegahan N+1

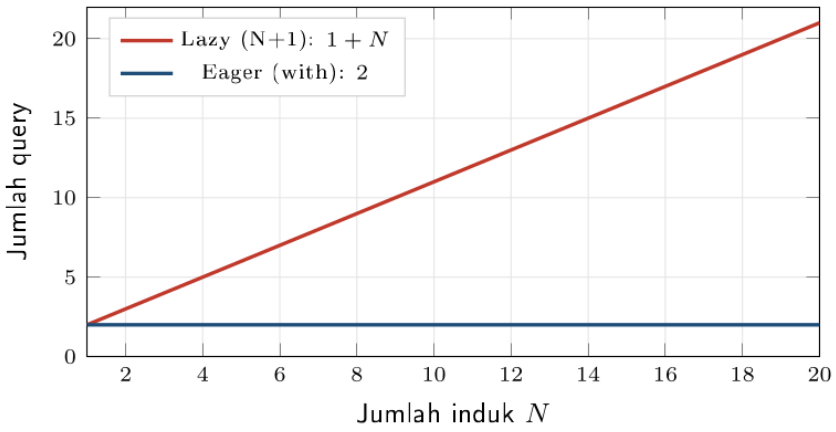
Setelah model dan relasi tersedia, Anda perlu memilih cara memuat data dengan efisien. Salah satu masalah umum pada ORM adalah *N+1 query*.

3.6.1 Memahami N+1 Query

Misalkan halaman menampilkan *N* menu dan nama tenant untuk setiap menu. Jika Anda mengambil menu terlebih dahulu, lalu memanggil `$menu->tenant` di dalam perulangan tanpa memuat relasi sebelumnya, aplikasi menjalankan sejumlah query seperti pada Persamaan 3.1.

$$Q_{\text{lazy}} = 1 + N, \qquad Q_{\text{eager}} = 2 \tag{Persamaan 3.1}$$

dengan ruas kiri adalah jumlah query ketika relasi dimuat secara *lazy*: angka 1 untuk mengambil daftar menu, ditambah *N* query untuk mengambil tenant setiap menu. Ruas kanan berlaku untuk satu relasi sederhana yang dimuat secara *eager*, yaitu satu query untuk menu dan satu query untuk seluruh tenant terkait.



Gambar 3.5. Perbandingan jumlah query pada lazy loading dan eager loading satu relasi.

Sumber: diadaptasi penulis, data ilustratif.

Jumlah menu	Lazy loading	Eager loading satu relasi
5	6	2
20	21	2
50	51	2
100	101	2

Tabel 3.8. Ilustrasi jumlah query untuk satu relasi sederhana.

Tabel tersebut adalah ilustrasi untuk satu relasi sederhana. Jika Anda memuat beberapa relasi, relasi bertingkat, atau relasi polimorfik, jumlah query dapat berbeda.

3.6.2 Menggunakan Eager Loading

Kode 3.8. Mengambil menu beserta tenant

```php
$menus = Menu::query()
    ->with('tenant')
    ->where('tersedia', true)
    ->orderBy('nama')
    ->get();

foreach ($menus as $menu) {
    echo $menu->nama . ' - ' . $menu->tenant->nama;
}
```

`with('tenant')` meminta Eloquent memuat relasi tenant sebelum data digunakan dalam perulangan. Dengan demikian, akses `$menu->tenant->nama` tidak memicu query tambahan untuk setiap baris menu.

Gunakan *eager loading* ketika Anda sudah mengetahui relasi akan diperlukan untuk seluruh atau sebagian besar data yang ditampilkan. Jangan memuat relasi yang tidak dipakai karena hal itu menambah penggunaan memori dan beban query.

3.6.3 Memilih Query Builder atau Eloquent

Gunakan Eloquent ketika Anda bekerja dengan model domain dan relasi yang jelas. Gunakan Query Builder ketika Anda membutuhkan rekap, agregasi, atau join yang lebih langsung.

Kebutuhan	Pendekatan yang lazim
Menampilkan detail satu menu beserta tenant dan kategori	Eloquent dengan relasi
Menampilkan daftar menu dengan relasi yang dipakai di view	Eloquent dengan eager loading
Menghitung total stok atau total penjualan per tenant	Query Builder
Menjalankan query sangat khusus	SQL mentah dengan parameter binding dan pengujian

Tabel 3.9. Memilih pendekatan akses data sesuai kebutuhan.

Keputusan tidak harus bersifat mutlak. Dalam satu aplikasi, Eloquent dan Query Builder dapat digunakan berdampingan sesuai kebutuhan.

3.7 Data Uji, Seeder, dan Verifikasi

Skema yang berhasil dimigrasikan belum tentu siap dipakai. Anda perlu mengisi data representatif dan memeriksa apakah aturan skema serta query benar-benar bekerja.

3.7.1 Factory dan Seeder

Factory membuat pola data uji untuk model. *Seeder* mengatur data awal atau data demo yang perlu disiapkan.

Kode 3.9. Factory menu

```php
<?php

namespace Database\Factories;

use App\Modules\Catalog\Models\Menu;
```

```php
use Illuminate\Database\Eloquent\Factories\Factory;

class MenuFactory extends Factory
{
    protected $model = Menu::class;

    public function definition(): array
    {
        return [
            'nama'      => fake()->words(2, true),
            'harga'     => fake()->numberBetween(8000, 30000),
            'stok'      => fake()->numberBetween(0, 50),
            'tersedia' => true,
        ];
    }
}
```

Factory tersebut belum menetapkan `tenant_id` dan `category_id`. Nilai tersebut perlu diberikan oleh seeder atau *state* factory karena migration `menus` mensyaratkan keduanya.

Kode 3.10. Seeder data demo

```php
public function run(): void
{
    $tenant = Tenant::firstOrCreate(
        ['kode' => 'KNT-001'],
        ['nama' => 'Kantin Teknik']
    );

    $category = MenuCategory::firstOrCreate([
        'tenant_id' => $tenant->id,
        'nama'      => 'Minuman',
    ]);

    Menu::factory()
        ->count(10)
        ->create([
            'tenant_id'   => $tenant->id,
            'category_id' => $category->id,
        ]);
}
```

Contoh tersebut menunjukkan bahwa data induk dibuat lebih dahulu, kemudian menu dibuat dengan foreign key yang valid. Namun, bagian `Menu::factory()->count(10)->create()` masih akan membuat menu baru setiap seeder dijalankan. Karena itu, contoh tersebut **tidak boleh** disebut idempoten.

Jika seeder perlu idempoten, gunakan identitas data yang stabil dan metode seperti `firstOrCreate()` atau `updateOrCreate()` untuk setiap data yang harus unik. Alternatif lain adalah menggunakan basis data yang direset khusus untuk pengembangan atau pengujian.

> **Catatan.** Seeder data demo, seeder awal produksi, dan factory pengujian memiliki tujuan berbeda. Hindari menjalankan seeder demo pada basis data produksi tanpa kebijakan yang jelas.

3.7.2 Memverifikasi Data dan Migration

Setelah menjalankan migration dan seeder, lakukan pemeriksaan dasar.

Terminal pada root proyek

```
php artisan migrate
php artisan db:seed --class=DemoSeeder
```

Anda dapat memeriksa jumlah menu tersedia melalui Tinker.

```
Terminal pada root proyek
php artisan tinker --execute="echo App\Modules\Catalog\Models\Menu::where('tersedia', true)->count();"
```

Pemeriksaan ini tidak menggantikan pengujian otomatis, tetapi membantu memastikan data dasar dapat dibuat dan diakses.

3.7.3 Indeks dan Rencana Eksekusi

Indeks adalah struktur tambahan yang membantu basis data menemukan baris tanpa selalu memindai seluruh tabel. Indeks berguna untuk kolom yang sering dipakai pada kondisi `WHERE`, penggabungan `JOIN`, pengurutan tertentu, dan aturan unik.

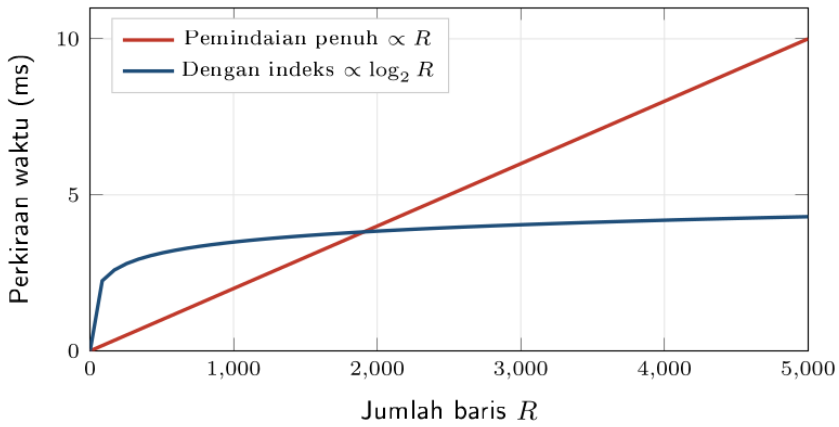
Indeks komposit `unique(['tenant_id', 'nama'])` pada tabel `menus` membantu dua tujuan: menegakkan keunikan nama menu dalam tenant, dan mendukung query yang mencari menu berdasarkan `tenant_id` dan `nama`.

Model konseptual pada Persamaan 3.2 membantu membedakan pemindaian penuh dan pencarian dengan indeks B-tree.

$$t_{\text{scan}} \propto R, \qquad t_{\text{index}} \propto \log_2 R + m$$

(Persamaan 3.2)

dengan *R* adalah jumlah baris yang dipertimbangkan, *k* adalah jumlah baris hasil yang cocok, sedangkan kedua ruas merupakan pendekatan biaya pencarian — bukan ukuran waktu nyata.



Gambar 3.6. Model konseptual biaya pemindaian penuh dibanding pencarian berindeks.

Sumber: diadaptasi penulis, data ilustratif.

Nilai tersebut hanya membantu membangun intuisi bahwa pencarian melalui struktur indeks dapat mengurangi jumlah langkah pencarian dibanding pemindaian seluruh tabel. Kinerja nyata tetap dipengaruhi oleh selektivitas data, urutan kolom pada indeks komposit, statistik basis data, ukuran hasil, cache, dan jenis query.

Untuk memeriksa rencana eksekusi, gunakan `EXPLAIN` melalui alat yang sesuai dengan lingkungan Anda.

```
Klien basis data MariaDB
EXPLAIN
SELECT *
FROM menus
WHERE tenant_id = 1
  AND nama = 'Kopi';
```

Periksa apakah rencana eksekusi menunjukkan indeks yang relevan. Nama indeks yang ditampilkan dapat berbeda dari nama kolom karena indeks komposit dapat diberi nama otomatis atau nama khusus saat dibuat.

Peringatan. Indeks mempercepat banyak operasi baca, tetapi menambah kebutuhan penyimpanan dan biaya pada operasi tulis seperti `INSERT`, `UPDATE`, dan `DELETE`. Buat indeks berdasarkan pola query yang benar-benar digunakan, bukan pada setiap kolom.

[PERLU VERIFIKASI: format keluaran EXPLAIN dan nama indeks pada versi Laravel serta MariaDB yang digunakan proyek]

3.8 Studi Kasus Terintegrasi: Katalog Menu Multi-Tenant

Bagian ini menunjukkan bagaimana unsur-unsur bab saling terhubung dalam fitur katalog menu tenant.

3.8.1 Kebutuhan Fitur

Seorang operator tenant harus dapat menambahkan menu. Pelanggan kemudian dapat melihat menu yang tersedia dari tenant yang sesuai. Sistem harus memastikan:

- menu terkait dengan tenant yang benar;
- menu memiliki kategori yang valid;
- nama menu tidak duplikat dalam tenant yang sama;
- harga disimpan sebagai Rupiah bulat;
- daftar menu tidak memicu N+1 query ketika menampilkan tenant atau kategori;
- data demo dapat dibuat untuk pengujian.

3.8.2 Keputusan Implementasi

Kebutuhan	Keputusan implementasi
Identitas menu	Primary key id pada tabel menus
Pemilik menu	Foreign key tenant_id
Pengelompokan menu	Foreign key category_id
Pencegahan nama duplikat	Unique index ['tenant_id', 'nama']
Harga menu	unsignedBigInteger('harga')
Status ketersediaan	boolean('tersedia')
Akses model	Model Menu dengan relasi tenant() dan category()
Daftar menu efisien	Eloquent dengan with(['tenant', 'category'])
Data demo	Seeder membuat tenant dan kategori sebelum menu

Tabel 3.10. Pemetaan kebutuhan fitur katalog ke keputusan implementasi.

3.8.3 Query Daftar Menu

Kode 3.11. Query daftar menu untuk halaman katalog

```php
$menus = Menu::query()
    ->with(['tenant', 'category'])
    ->where('tersedia', true)
    ->orderBy('nama')
    ->get();
```

Query tersebut dipilih karena halaman akan menampilkan data menu, nama tenant, dan nama kategori. Relasi dimuat sejak awal agar view tidak memicu query tambahan per menu.

3.8.4 Analisis Risiko

Beberapa risiko yang perlu diperiksa dalam implementasi ini adalah:

- `tenant_id` diterima langsung dari request tanpa memeriksa tenant operator;
- migration menggunakan `cascadeOnDelete()` pada data yang seharusnya dipertahankan untuk riwayat;
- factory membuat data tanpa foreign key yang wajib;
- seeder dijalankan berulang dan menghasilkan duplikasi;
- halaman katalog mengakses relasi dalam perulangan tanpa eager loading;
- indeks dibuat tanpa memeriksa pola query dan rencana eksekusi.

Dengan meninjau risiko tersebut sejak awal, Anda dapat mengurangi masalah integritas data dan performa sebelum aplikasi menjadi lebih kompleks.

Rangkuman Bab

- Implementasi basis data di Laravel perlu dilihat sebagai proses yang terhubung: ERD, migration, model, relasi, query, data uji, dan verifikasi.
- Konfigurasi koneksi dipisahkan melalui environment dan konfigurasi aplikasi. Kredensial tidak boleh disimpan pada repository.
- Eloquent, Query Builder, dan SQL mentah memiliki tingkat abstraksi berbeda. Parameter binding membantu mencegah injeksi SQL jika digunakan dengan benar.
- Migration menjaga versi skema. Foreign key, aturan unik, tipe data, dan kebijakan penghapusan merupakan keputusan desain yang penting.
- Uang dalam Rupiah dapat disimpan sebagai bilangan bulat untuk menghindari persoalan pecahan *float*.
- Model Eloquent mendefinisikan atribut yang dapat diisi, konversi tipe, dan relasi antarmodel.
- Relasi *many-to-many* dengan atribut transaksi dapat membutuhkan perlakuan sebagai entitas tersendiri, bukan hanya pivot sederhana.
- *Eager loading* membantu mencegah N+1 query, tetapi relasi yang dimuat harus tetap dipilih sesuai kebutuhan.
- Factory dan seeder membantu menyediakan data uji, tetapi harus memenuhi foreign key dan tidak boleh diklaim idempoten jika masih membuat data baru pada setiap eksekusi.
- Indeks perlu dibuat berdasarkan pola query dan diverifikasi melalui rencana eksekusi, bukan hanya berdasarkan asumsi.

Bab berikutnya melanjutkan pembahasan ke pengelolaan data multi-tenant: bagaimana konteks tenant ditetapkan, disaring, dan ditegakkan pada seluruh lapisan aplikasi.

Istilah Penting

Istilah	Makna ringkas
Migration	Kendali versi untuk struktur basis data
Eloquent ORM	Pemetaan tabel basis data menjadi objek model
Query Builder	Antarmuka fluent untuk menyusun query SQL
Foreign key	Kolom yang merujuk primary key pada tabel lain
Pivot table	Tabel penghubung dalam relasi many-to-many
Mass assignment	Pengisian banyak atribut model secara sekaligus
Eager loading	Pemuatan relasi lebih awal untuk menghindari N+1
Lazy loading	Pemuatan relasi saat relasi pertama kali diakses
N+1 query	Pola satu query awal ditambah query per data dalam perulangan
Index	Struktur basis data yang membantu pencarian baris
Seeder	Kelas untuk mengisi data awal atau data demo
Factory	Cetakan pembentukan data uji untuk model

Tabel 3.11. Istilah penting pada Bab 3.

Pertanyaan dan Latihan

A. Pemahaman

1. Jelaskan hubungan antara ERD, migration, model Eloquent, relasi, dan query.
2. Apa perbedaan utama antara Eloquent, Query Builder, dan SQL mentah?
3. Mengapa nilai uang Rupiah pada contoh bab ini disimpan sebagai `unsignedBigInteger`, bukan `float`?
4. Jelaskan perbedaan `cascadeOnDelete()`, `restrictOnDelete()`, dan `nullOnDelete()`.
5. Mengapa `$fillable` perlu dirancang secara hati-hati pada aplikasi multi-tenant?

B. Penerapan

6. Buat migration tabel `menu_categories` yang memiliki relasi ke tenant dan memastikan nama kategori unik dalam satu tenant.
7. Buat model `MenuCategory` dengan relasi `tenant()` dan `menus()`.
8. Buat query Query Builder untuk menghitung jumlah menu tersedia per tenant.
9. Ubah query daftar menu agar memuat tenant dan kategori menggunakan eager loading.
10. Buat seeder yang menghasilkan satu tenant, dua kategori, dan lima menu per kategori. Pastikan setiap menu memiliki `tenant_id` dan `category_id` yang valid.

C. Analisis

11. Sebuah halaman mengambil N menu dan menampilkan satu relasi tenant untuk setiap menu. Hitung jumlah query pada *lazy loading* dan *eager loading* berdasarkan Persamaan 3.1 untuk $N = 75$. Berapa selisihnya?

12. Halaman lain menampilkan 40 pesanan, lalu mengakses relasi pelanggan dan tenant pada setiap pesanan. Jelaskan mengapa jumlah query tidak dapat langsung dianggap selalu dua ketika eager loading digunakan. Rancang strategi eager loading yang sesuai.
13. Analisis risiko penggunaan `cascadeOnDelete()` pada tabel `tenants` yang memiliki relasi ke menu, pesanan, dan pembayaran.
14. Sebuah aplikasi menerima `tenant_id` dari formulir penambahan menu. Jelaskan risiko yang mungkin terjadi dan rancang cara menentukan tenant secara lebih aman.
15. Sebuah query sering mencari menu dengan kondisi `WHERE tenant_id = ? AND nama = ?`. Jelaskan mengapa indeks komposit (`tenant_id, nama`) relevan. Bagaimana pertimbangan Anda jika query yang dominan justru mencari berdasarkan nama saja?

Daftar Pustaka

Buku

Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.

Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill Education.

Stauffer, M. (2023). *Laravel: Up & Running* (3rd ed.). O'Reilly Media.

Artikel Jurnal dan Prosiding

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387. https://doi.org/10.1145/362384.362685

Comer, D. (1979). The ubiquitous B-tree. *ACM Computing Surveys*, 11(2), 121–137. https://doi.org/10.1145/356770.356776

Dokumentasi dan Standar (Daring)

Laravel LLC. (2026). *Database: Migrations, Query Builder, Eloquent ORM — Laravel 13.x Documentation*. Diakses dari https://laravel.com/docs/13.x/eloquent

MariaDB Foundation. (2026). *MariaDB Server Documentation — Optimization and Indexes*. Diakses dari https://mariadb.com/kb/en/optimization-and-indexes/

PHP Group. (2026). *PHP Data Objects (PDO)*. Diakses dari https://www.php.net/manual/en/book.pdo.php

Sumber Proyek (Dokumen Internal)

Politeknik Negeri Banyuwangi. *SRS Aplikasi Kantin Multi-Tenant (ISO/IEC/IEEE 29148), versi 2 — §3.5 dan Lampiran B*. Dokumen proyek.

Politeknik Negeri Banyuwangi. *ERD Aplikasi Kantin Multi-Tenant (Baseline Fase 0)*. Artefak desain.

Politeknik Negeri Banyuwangi. *Modul Praktikum Kantin Multi-Tenant Laravel 13 (Mentor Tutorial)*. Dokumen praktikum.

Catatan: seluruh data numerik, diagram, dan hasil pengujian pada bab ini bersifat ilustratif untuk keperluan pembelajaran.